CPE112HDS — Module1 — Linear Data Structures

Lecture5: Basic Sorting Algorithm

Dr. Prapong Prechaprapranwong

Computer Engineering

King Mongkut's University of Technology Thonburi

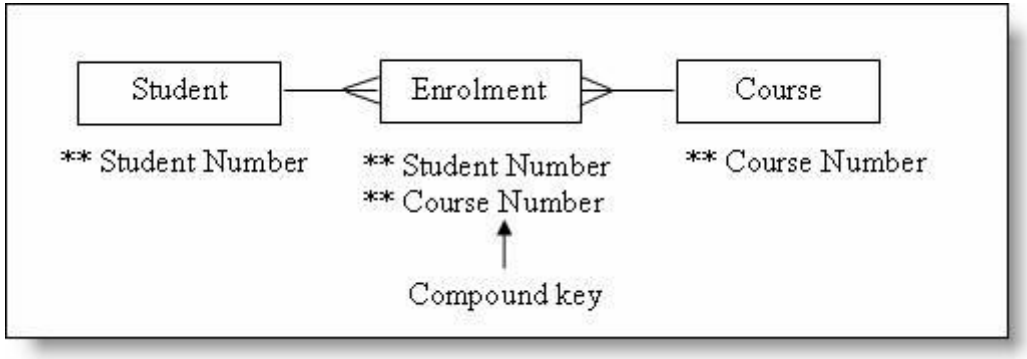
Content

- Searching
- Sorting
  - Bubble Sort
  - Selection Sort
  - Insertion Sort

S e a r c h i n g is the process of selecting particular information from a collection of data based on specific criteria.

A key or **A Target** is a unique value used to identify the data elements of a collection.

- In collections containing simple types such as integers or reals, the values themselves are the keys.
- For collections of complex types, a specific data component has to be identified as the key.
- In some instances, a key may consist of multiple components, which is also known as a **compound key**.

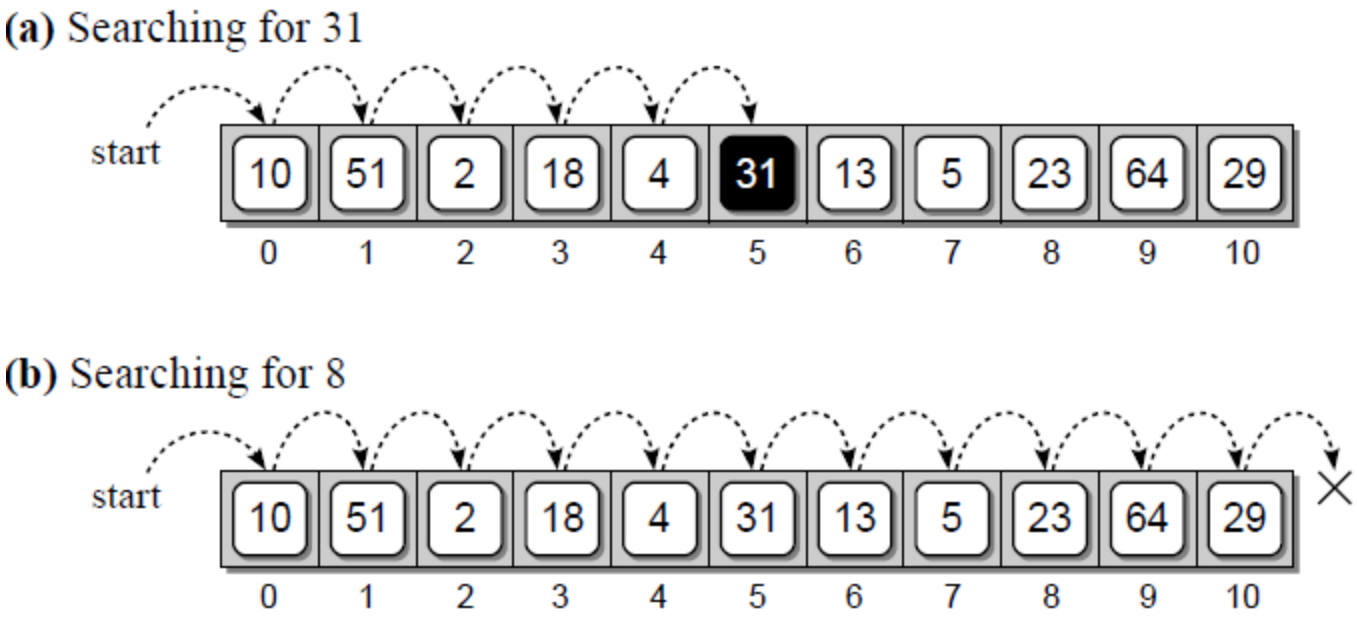


Sequential search or **Linear search** is the standard approach to search for a target value is to use a loop to examine every element, until either finding the target or exhausting the data set

```
def sequential_search(seq,key):
    for i in range(len(seq))
        if seq[i] == key: return True
return False
```

```
def sequential_search(seq, key):
    for item in seq:
        if item == key: return True
    return False
```

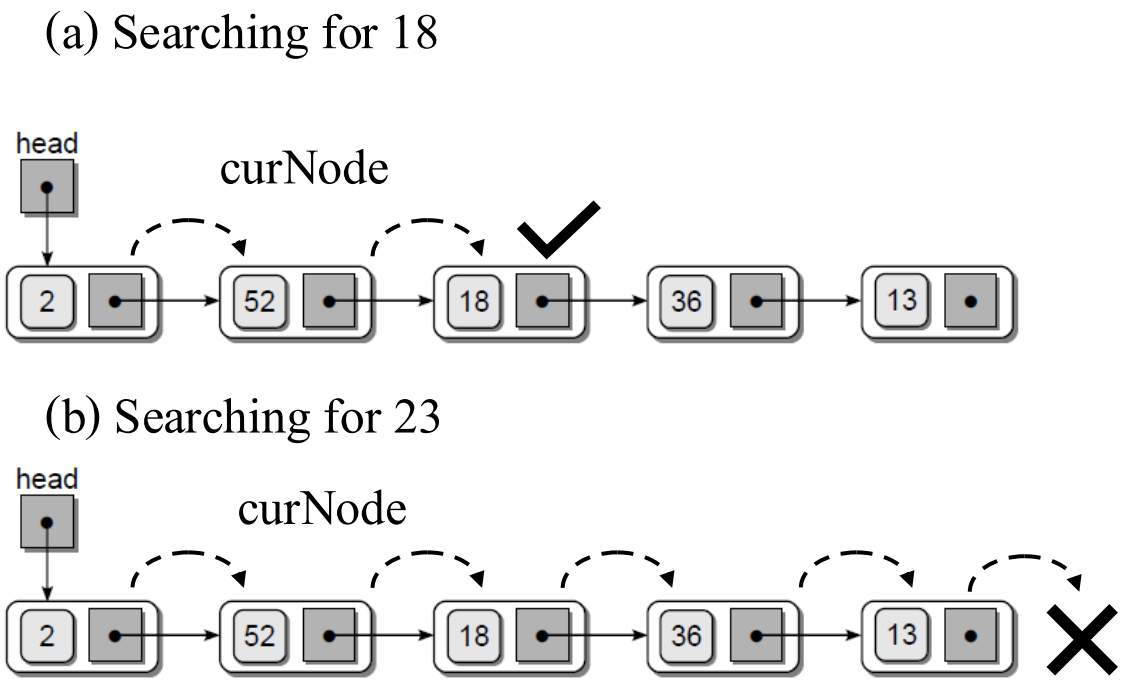
Searching in array or python list



Sequential search in linked list

```
def search(self,target):
    curNode = self._head
    while curNode is not None and curNode._item != target:
        curNode = curNode._next
    return curNode is not None
```

Searching in linked list

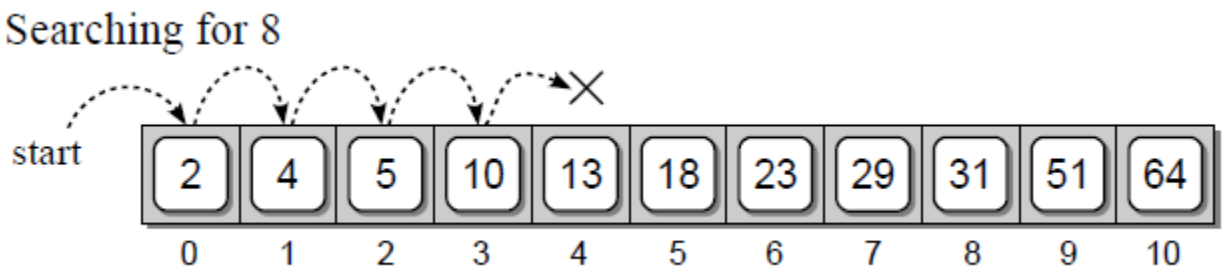


Assuming the sequence contains n items, the sequential search or linear search has a worst case time of O(n).

Searching a sorted sequence

A linear search on a sorted sequence works in the same fashion as that for the unsorted sequence, with one exception. It's possible to terminate the search early when the value is not in the sequence instead of always having to perform a complete traversal.

```
def ordered_sequential_search(seq, key):
    item = 0
    for item in seq:
        if item > key: return False
        if item == key: return True
    return False
```



Sorting is the process of arranging or ordering a collection of items such that each item and its successor satisfy a prescribed relationship.

- Bubble Sort
- Selection Sort
- Insertion Sort
- Count Sort
- Heap Sort
- Merge Sort
- Quick Sort
- Radix Sort
- Bucket Sort

Quadratic

Linear

Loglinear

Bubble Sort

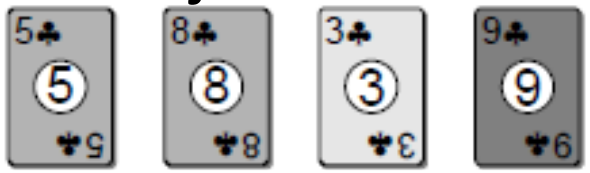
```
# Sorts a sequence in ascending order using the bubble sort algorithm.
def bubbleSort( seq ):
        n = len( seq ) - 1
        # Perform n-1 bubble operations on the sequence
        for i in range( n , 0 , -1) :
        # Bubble the largest item to the end
                for j in range(i) :
                  if seq[j] > seq[j + 1] : # swap the j and j+1 items
                        tmp = seq[j]
                        seq[j] = seq[j + 1]
                        seq[j + 1] = tmp
```

n = 3

i = 3

j = 0..2

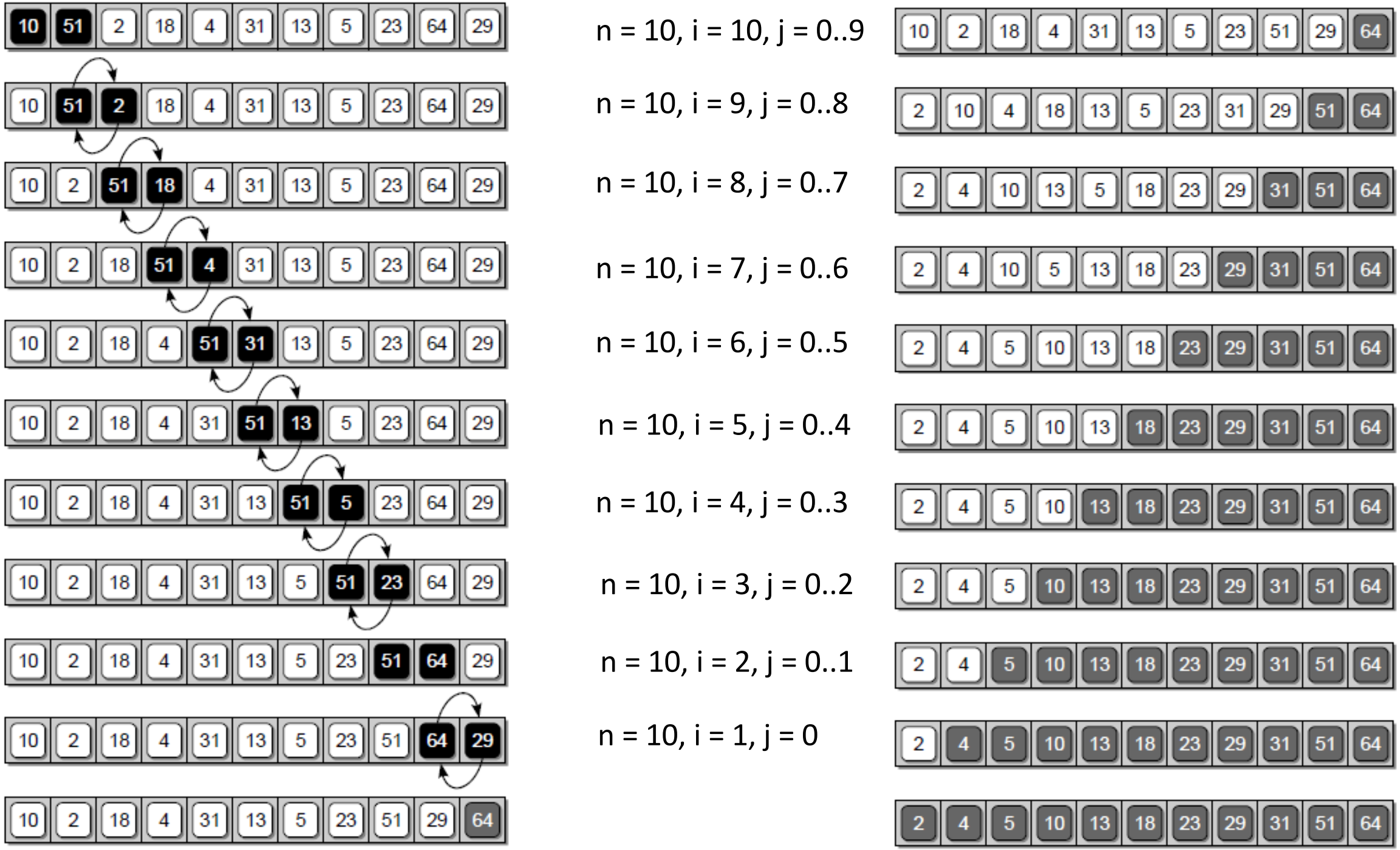
i = 2 j = 0..1



```
# Sorts a sequence in ascending order using the bubble sort algorithm.
def bubbleSort( seq ):
        n = len( seq ) - 1
        # Perform n-1 bubble operations on the sequence
        for i in range( n , 0 , -1) :
        # Bubble the largest item to the end
                for j in range(i) :
                  if seq[j] > seq[j + 1] : # swap the j and j+1 items
                        tmp = seq[j]
                        seq[j] = seq[j + 1]
                        seq[j + 1] = tmp
```

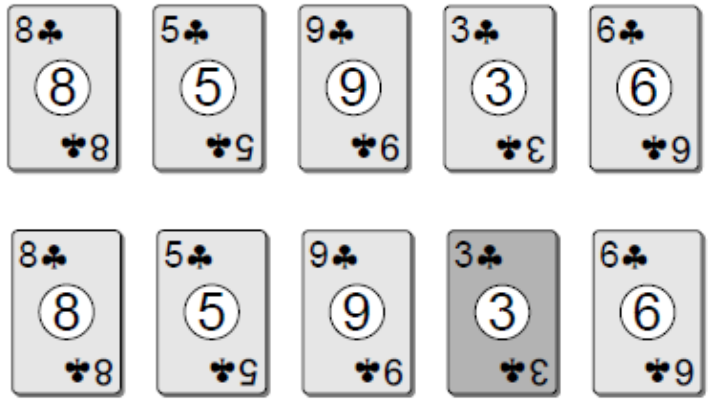
i = i j = 0

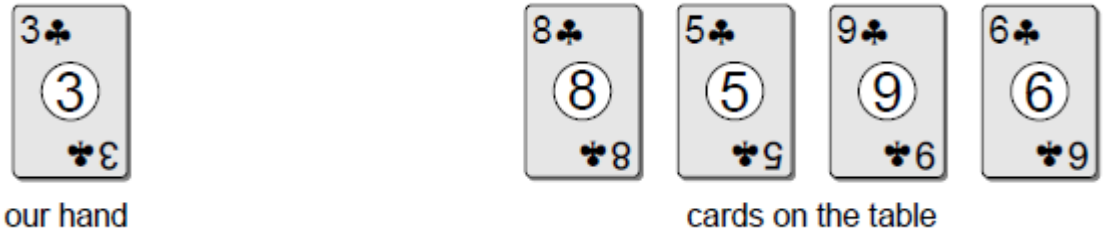
Bubble sort is considered one of the most inefficient sorting algorithms due to the total number of swaps required.


10 51 2 18 4 31 13 5 23 64 29
n = 10, i = 10, j = 0..9
10 2 18 4 31 13 5 23 51 29 64
10 51 2 18 4 31 13 5 23 64 29
n = 10, i = 9, j = 0..8
2 10 4 18 13 5 23 31 29 51 64
10 2 51 18 4 31 13 5 23 64 29
n = 10, i = 8, j = 0..7
2 4 10 13 5 18 23 29 31 51 64
10 2 18 51 4 31 13 5 23 64 29
n = 10, i = 7, j = 0..6
2 4 10 5 13 18 23 29 31 51 64
10 2 18 4 51 31 13 5 23 64 29
n = 10, i = 6, j = 0..5
2 4 5 10 13 18 23 29 31 51 64
10 2 18 4 31 51 13 5 23 64 29
n = 10, i = 5, j = 0..4
2 4 5 10 13 18 23 29 31 51 64
10 2 18 4 31 13 51 5 23 64 29
n = 10, i = 4, j = 0..3
2 4 5 10 13 18 23 29 31 51 64
10 2 18 4 31 13 5 51 23 64 29
n = 10, i = 3, j = 0..2
2 4 5 10 13 18 23 29 31 51 64
10 2 18 4 31 13 5 23 51 64 29
n = 10, i = 2, j = 0..1
2 4 5 10 13 18 23 29 31 51 64
10 2 18 4 31 13 5 23 51 64 29
n = 10, i = 1, j = 0
2 4 5 10 13 18 23 29 31 51 64
10 2 18 4 31 13 5 23 51 29 64
2 4 5 10 13 18 23 29 31 51 64

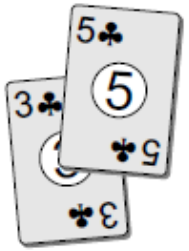
Selection Sort

Selection Sort improves on the bubble sort and works in a fashion similar to what a human may use to sort a list of values

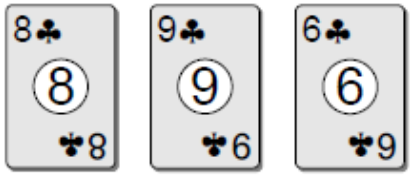




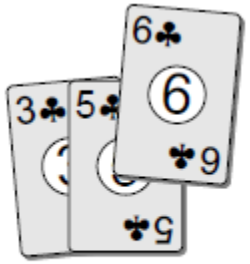
cards on the table



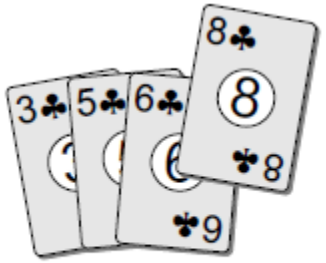
our hand



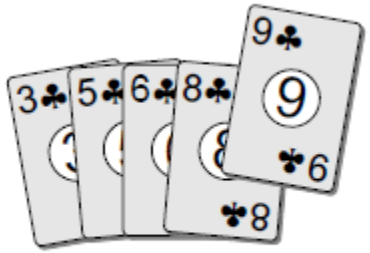
cards on the table



pick up the next smallest card (6)

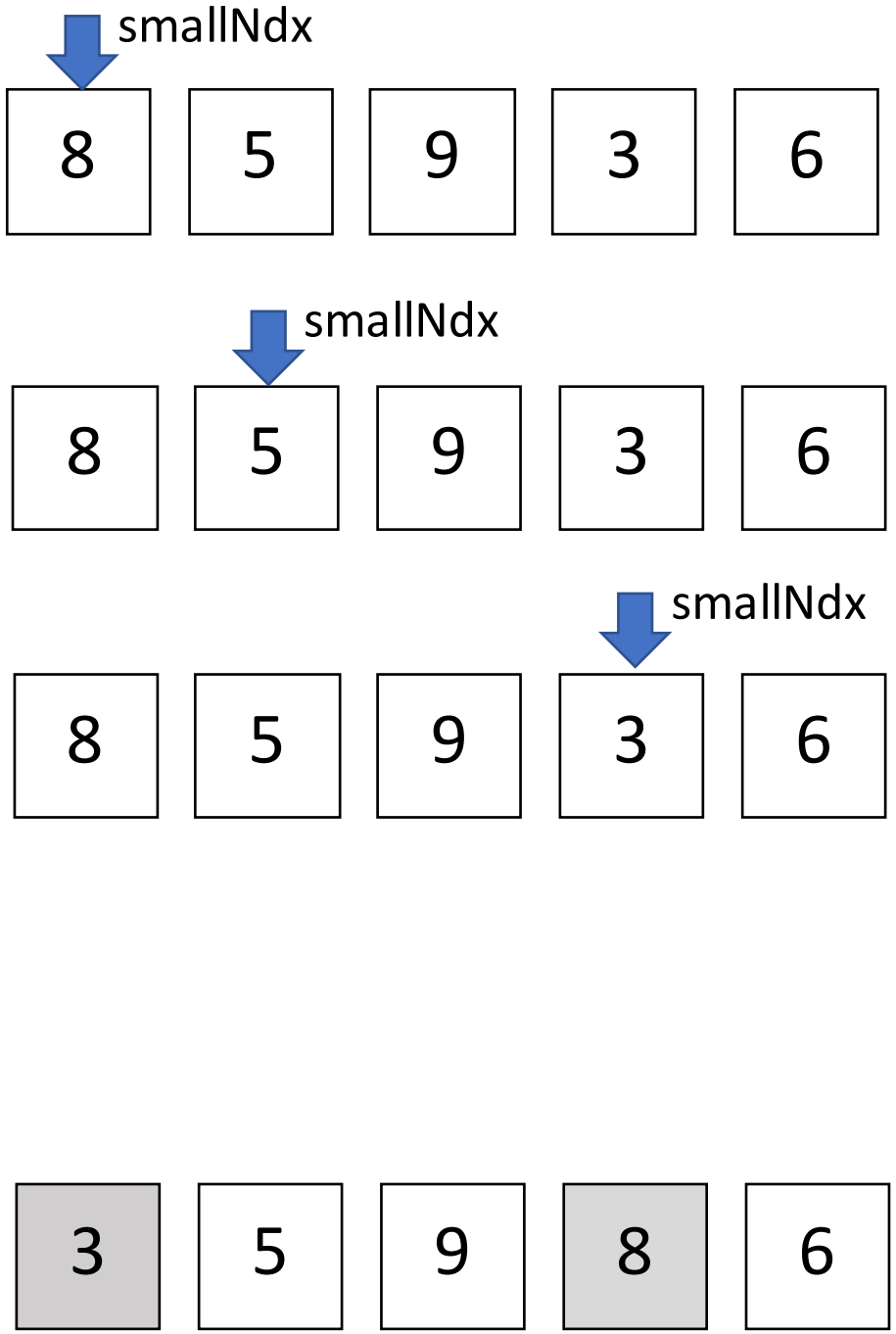


pick up the next smallest card (8)



pickup the last card (9)

the resulting hand



n = 5

i = 0

smallNdx = 0

j = 4

smallNdx = 3

swap

```
def selectionSort( seq ):
    n = len( seq )
    for i in range( n - 1 ):
        smallNdx = i
        for j in range( i + 1, n ):
            if seq[j] < seq[smallNdx] :
                smallNdx = j
        if smallNdx != i :
            tmp = Seq[i]
            seq[i] = seq[smallNdx]
            seq[smallNdx] = tmp
    return seq
```

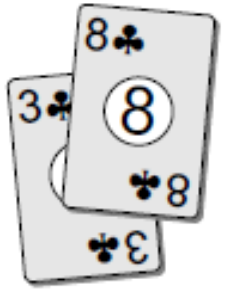
10
51
2
18
4
31
13
5
23
64
29

Insertion Sort

the deck

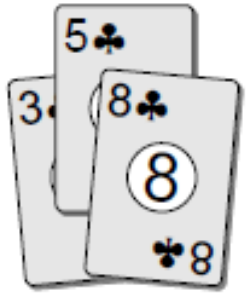
our hand

the deck



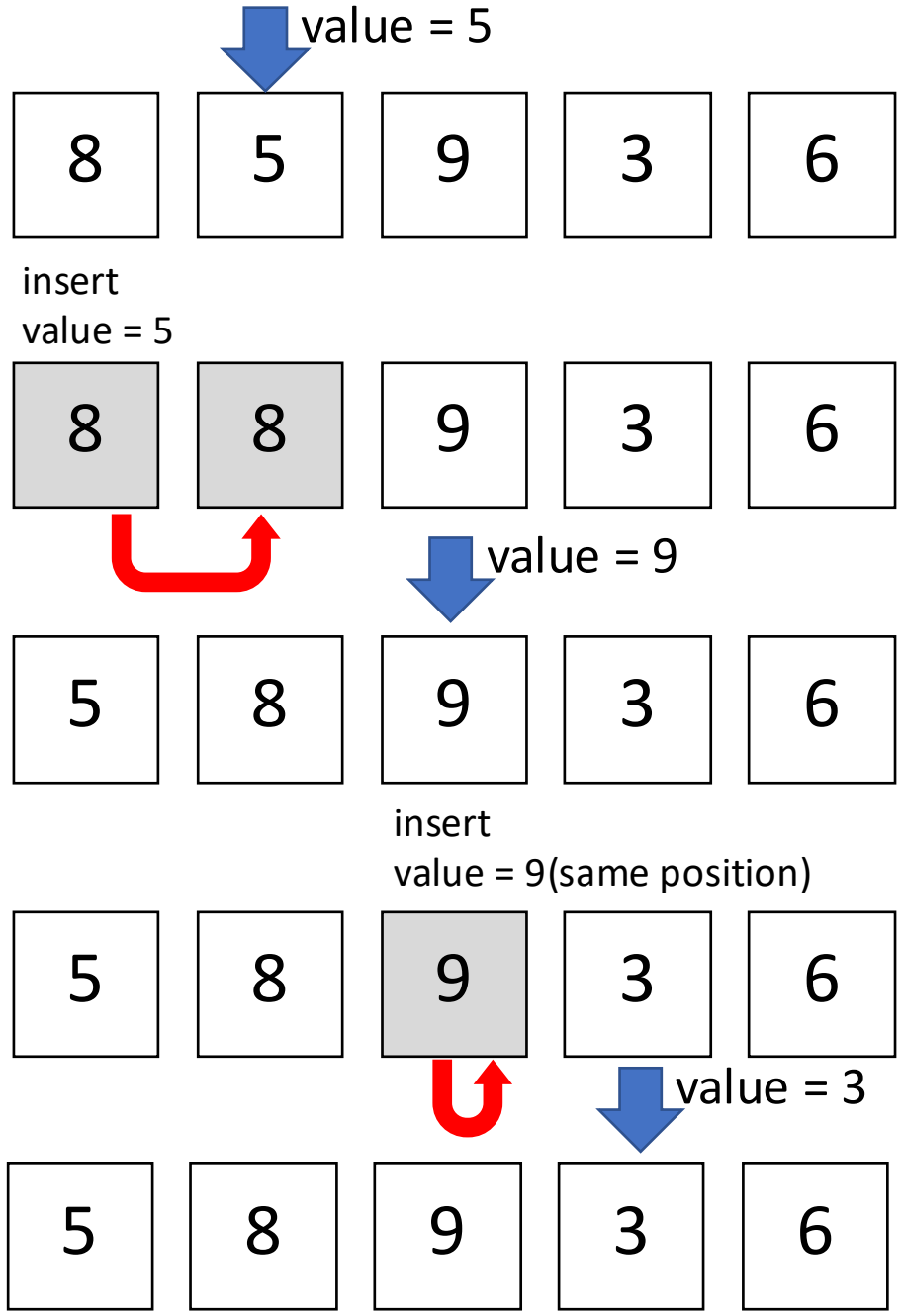
our hand

the deck

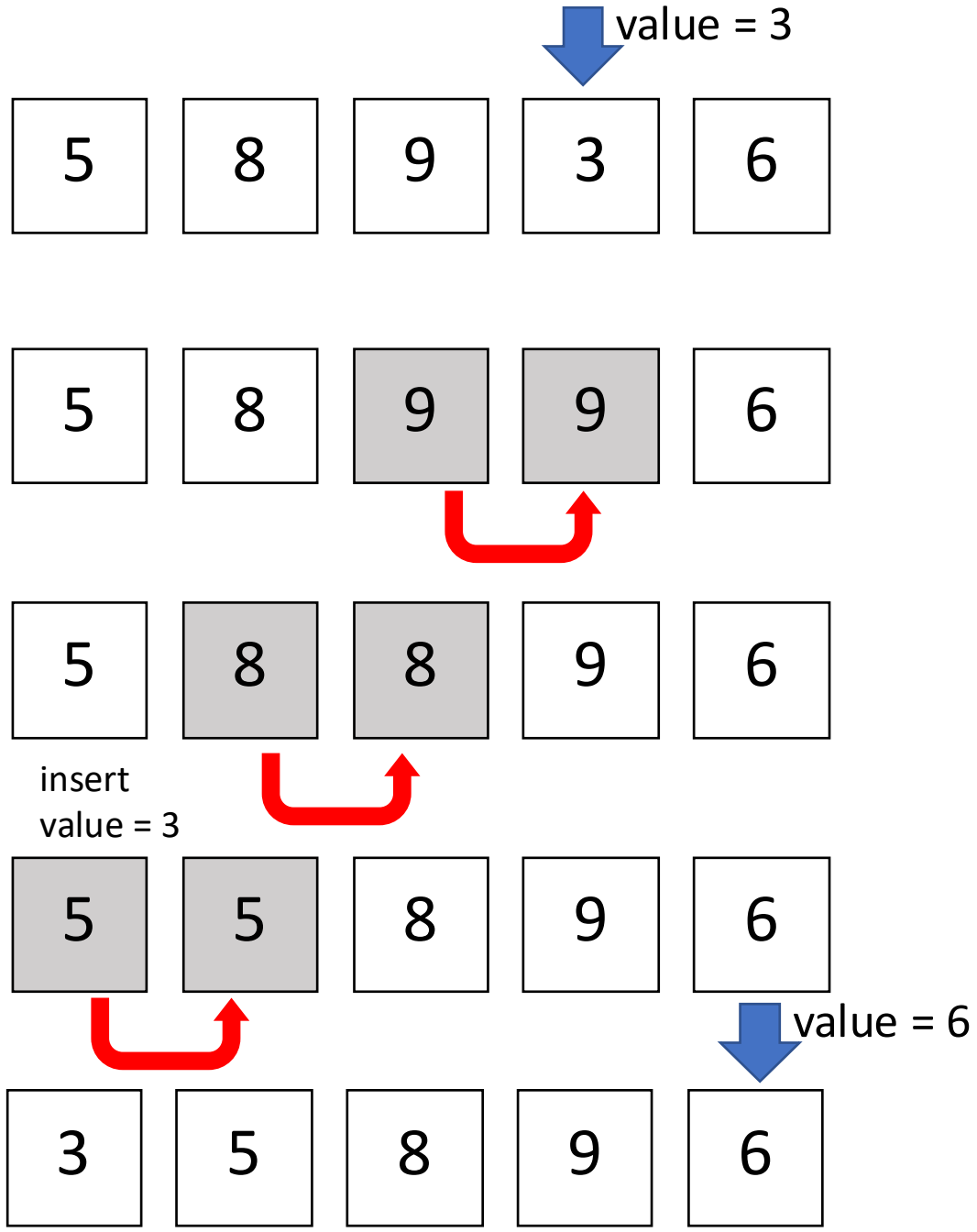


our hand

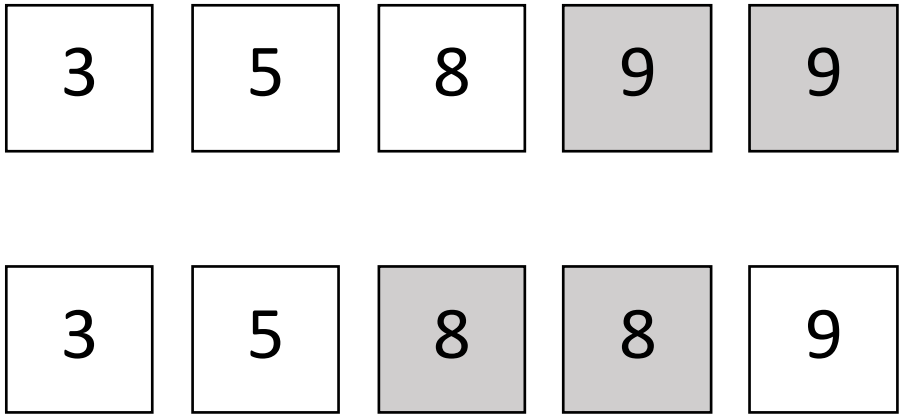
the deck

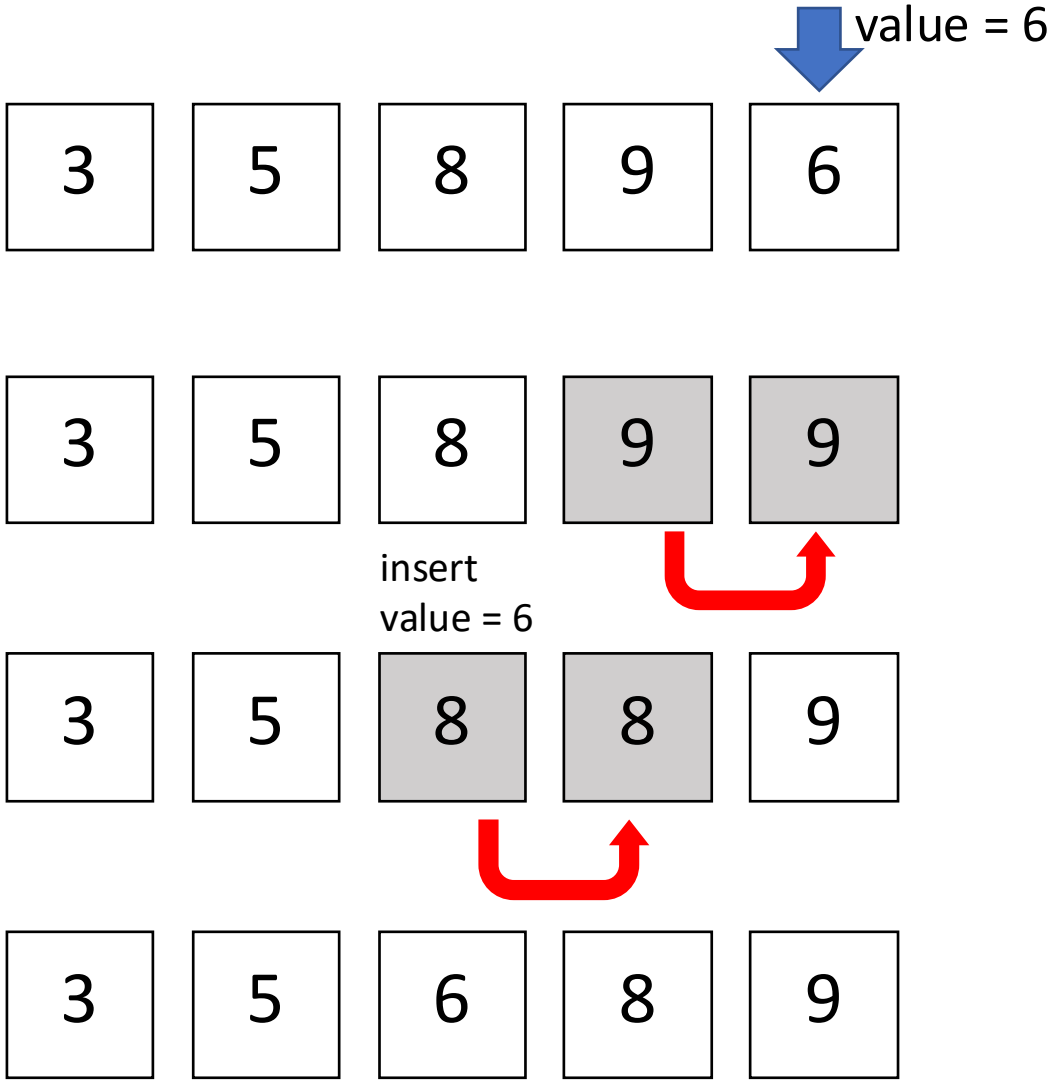



```
def insertionSort(seq):
    n = len(seq)
    for i in range(1,n):
        value = seq[i]
        pos = i
        while pos > 0 and value < seq[pos -1]:
            seq[pos] = seq[pos - 1]
            pos -= 1
        seq[pos] = value
    return seq
```

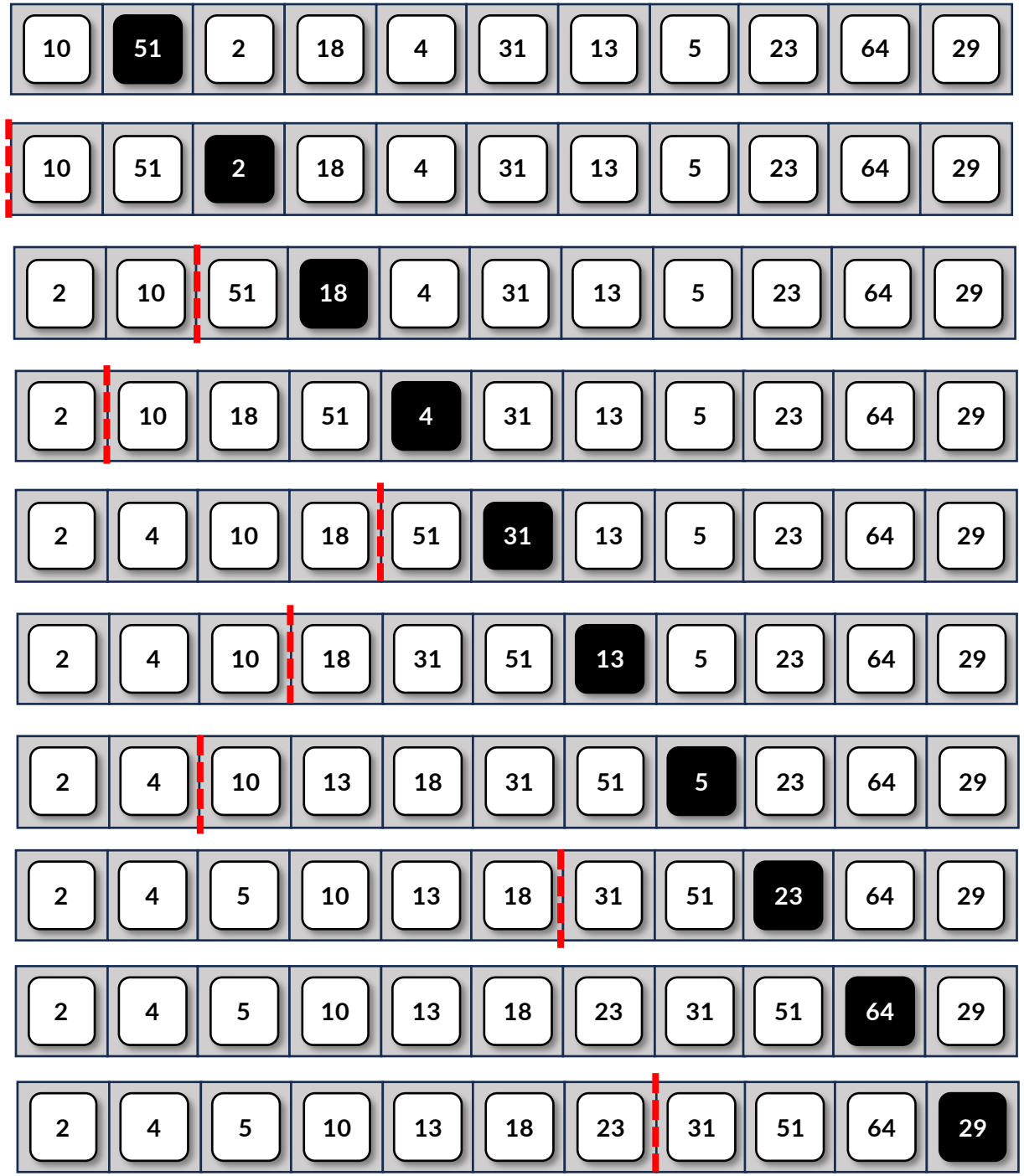


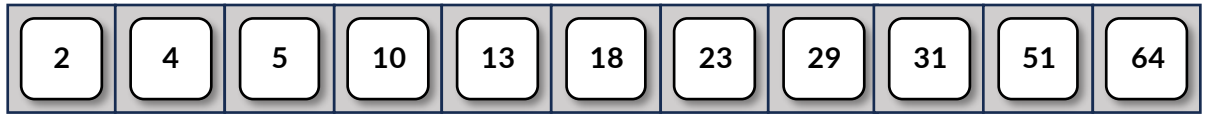

```
def insertionSort(seq):
    n = len(seq)
    for i in range(1,n):
        value = seq[i]
        pos = i
        while pos > 0 and value < seq[pos -1]:
            seq[pos] = seq[pos - 1]
            pos -= 1
        seq[pos] = value
    return seq
```






```
def insertionSort(seq):
    n = len(seq)
    for i in range(1,n):
        value = seq[i]
        pos = i
        while pos > 0 and value < seq[pos -1]:
            seq[pos] = seq[pos - 1]
            pos -= 1
        seq[pos] = value
    return seq
```


10 51 2 18 4 31 13 5 23 64 29
10 51 2 18 4 31 13 5 23 64 29
2 10 51 18 4 31 13 5 23 64 29
2 10 18 51 4 31 13 5 23 64 29
2 4 10 18 51 31 13 5 23 64 29
2 4 10 18 31 51 13 5 23 64 29
2 4 10 13 18 31 51 5 23 64 29
2 4 5 10 13 18 31 51 23 64 29
2 4 5 10 13 18 23 31 51 64 29
2 4 5 10 13 18 23 31 51 64 29


2 4 5 10 13 18 23 29 31 51 64

Summary

- **Searching** is the process of selecting specific information from a collection of data
- **Sequential search/Linear search** is the standard approach in searching to examine every element, until either finding the target or exhausting the data set
- The sequential search/linear search has a worst-case time complexity of O(n).
- A linear search can be improved by pre-sorting the sequence then we can early terminate the search if we know that the key is surely not in the sequence
- **Sorting** is the process of arranging or ordering an orderable collection of items
- **Bubble sort**, **Selection sort**, **Insertion sort** is categorized into **Quadratic sorting**

What should you know

- Understand searching process in each type of data structures
- How to implement Sorting algorithm with python
  - Bubble Sort
  - Selection sort
  - Insertion sort